

ENBC321/BIOE658I: Homework #9

Due: Wednesday 04/29/26 by 5:00 PM, beginning of the class

Submit your homework to ELMS, including Python code and the dataset

Overall points: 100

Note: Late homework may be submitted up to 24 hours after the due date, but a 50% penalty will be applied.

Note: Please aim to write clean, well-organized code that is easy for others to understand. Be sure to include comments or explanations where needed. Points may be deducted if this is not done.

Note: Please name your codes using the following format: “`problem1_fullname.ipynb`”. Start with the problem number and followed by your full name, e.g., “**problem1_jimmyazarnoosh**”.

Note: Run all cells and ensure there are no syntax errors. All results must be displayed in the notebook. (Syntax errors = zero points).

Note: Using generative AI for learning purposes is allowed. However, directly copying or closely replicating AI-generated content is strictly prohibited and will result in a grade of ZERO. Repeated violations may lead to more severe consequences.

Note: Submit **a single zip file** containing the following:

- The dataset you used
- Python code files in ***.ipynb**
- Your answers to the questions in PDF format

Problem 1. Deep Learning for Biological Data Analysis (50 points)

You are given synthetic data ([Bio data](#)) that simulates biological measurements from a lab experiment. Each sample represents gene expression levels (features) from a tissue sample. The gene expressions are continuous variables. You have two tasks:

- **Regression Task:** Predict the concentration of a certain protein based on the gene expressions (continuous output). You will use `MLPRegressor` for the regression.
- **Classification Task:** Predict whether a sample is cancerous or healthy (binary output) based on the gene expressions. You will use `MLPClassifier` for the classification.

Tasks:

Regression with `MLPRegressor`

- Load the data from `bio_data.csv`.
- Split the dataset into features (X) and target (y), Protein_Concentration.
- **Split the data** into training and test sets (80% train, 20% test).
- Scale the features using `StandardScaler` before training the model.
- **Train an `MLPRegressor`** to predict protein concentration based on gene expression levels.
- Report **R^2 scores** for the training and test sets.
- **Plot** the predicted vs. actual protein concentrations on the test set.

Classification with `MLPClassifier`

- Use the same features (X), but target (y), Cancer_Status.
- **Split the data** into training and test sets (80% train, 20% test).
- Scale the features using `StandardScaler` before training the model.
- **Train an `MLPClassifier`** to predict cancer status.
- Report the accuracy.
- Also, report a **confusion matrix** for the test set.

Conceptual questions:

1. In an MLP, what role do the activation functions play between layers? Why can't we just stack linear layers without activation functions?
2. Suppose your regression model has a very **high** R^2 score on training data but a **low** R^2 score on test data. What could this indicate about the model?
3. What are some hyperparameters of `MLPRegressor` and `MLPClassifier` that you could tune to try to improve model performance?

Problem 2. Image Classification with CNNs (50 points)

You will work with the **CIFAR-10 dataset**, which contains 60,000 32x32 color images in **10 classes**. Your goal is to build a convolutional neural network (CNN) to classify images into one of the 10 classes. You will also need to evaluate the model performance and predict the class of a given image. This problem is similar to the code discussed in the class `DL_CNN_Number.py`.

Tasks:

- Load the CIFAR-10 dataset using TensorFlow's `keras.datasets.cifar10`.
- Explore the dataset by printing the shape of the training and test data.
- Focus on the **'cat'** class (class index 3) and print the first 10 image indices from the test set that belong to this class. Select an image from the 'cat' class (class index = 3) and display it.
- Normalize the training and testing images so that pixel values are in the range of [0, 1].
- Reshape the image data to fit the input shape required by the convolutional neural network (32x32x3).
- Define a CNN model with the following architecture:
 - A convolutional layer with 32 filters, kernel size (3, 3), and ReLU activation.
 - A max-pooling layer with pool size (2, 2).
 - A convolutional layer with 64 filters, kernel size (3, 3), and ReLU activation.
 - Another max-pooling layer with pool size (2, 2).
 - A convolutional layer with 64 filters, kernel size (3, 3), and ReLU activation.
 - A flatten layer.
 - A dense layer with 64 neurons and ReLU activation.
 - A final dense layer with 10 neurons (corresponding to the 10 classes) and a softmax activation function.
- Compile the model using the **Adam** optimizer and **sparse categorical cross-entropy** loss function. Monitor the **accuracy** during training.
- Train the model on the training data for **5 epochs** with a validation split of 0.1.
- Evaluate the model on the test data and print the test accuracy.
- Predict the class of the image you selected from the 'cat' class in Task 1.
- Display the image with a title showing the predicted and actual labels.